Principles of Database System - Individual Project

Student Name: RITA RABAYA JAHAN

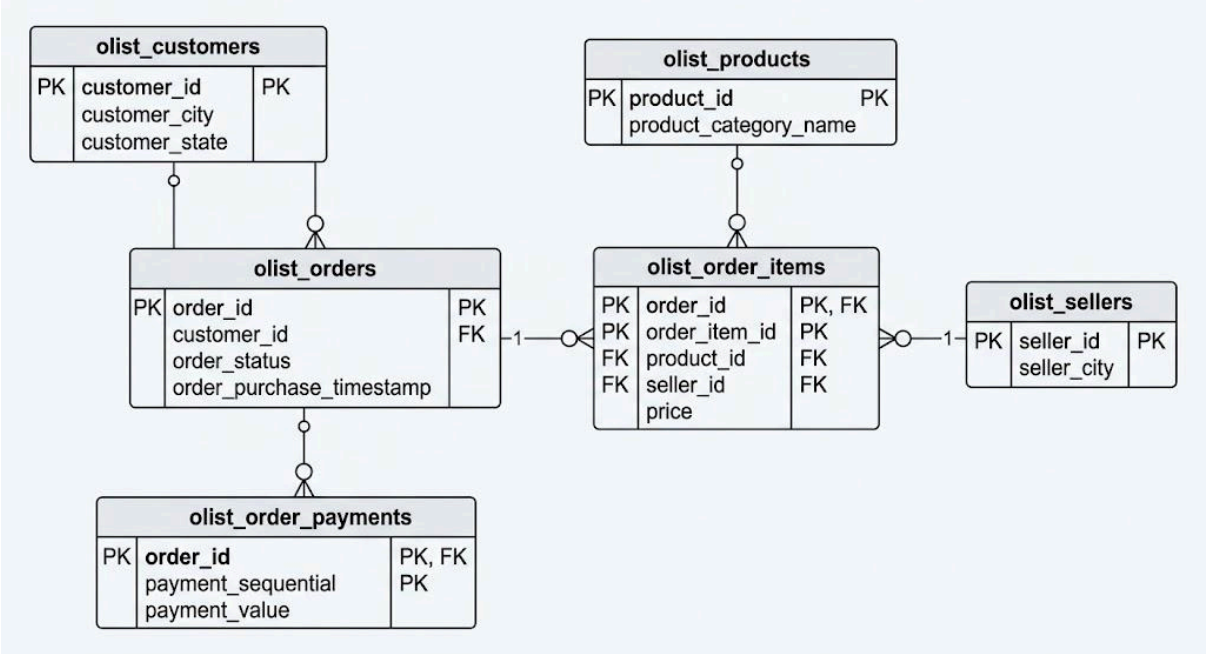
ID: 2511632118

Dataset: Brazilian E-Commerce (Olist)

📄 SECTION 1: Introduction

"For my final project, I selected the Brazilian E-Commerce (Olist) dataset from Kaggle. I built a PostgreSQL relational database, cleaned and loaded the data using Python (ETL), and wrote complex SQL queries to analyze business performance. I also optimized the database by creating indexes."

📄 SECTION 2: Task 1 - ER Diagram



"This ER diagram models the Olist database. The central entity is `olist_orders`. One customer can have many orders (1:N). Each order can have multiple items (1:N). Each item links to exactly one product and one seller. Payments are linked to orders (1:N). Primary keys (PK) uniquely identify rows, and Foreign keys (FK) create the links."

📄 SECTION 3: Task 2 - Database Schema

```
1. Customers Table
CREATE TABLE olist_customers (
    customer_id VARCHAR(50) PRIMARY KEY,
    customer_unique_id VARCHAR(50) NOT NULL,
    customer_zip_code_prefix INT,
    customer_city VARCHAR(100),
    customer_state VARCHAR(10)
);

-- 2. Sellers Table
CREATE TABLE olist_sellers (
    seller_id VARCHAR(50) PRIMARY KEY,
    seller_zip_code_prefix INT,
    seller_city VARCHAR(100),
    seller_state VARCHAR(10)
);

-- 3. Products Table
CREATE TABLE olist_products (
    product_id VARCHAR(50) PRIMARY KEY,
    product_category_name VARCHAR(100),
    product_name_length INT,
    product_description_length INT,
    product_photos_qty INT,
    product_weight_g NUMERIC,
    product_length_cm NUMERIC,
    product_height_cm NUMERIC,
    product_width_cm NUMERIC
);

-- 4. Orders Table
CREATE TABLE olist_orders (
    order_id VARCHAR(50) PRIMARY KEY,
    customer_id VARCHAR(50),
    order_status VARCHAR(20),
    order_purchase_timestamp TIMESTAMP,
    order_approved_at TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP,
    CONSTRAINT fk_customer FOREIGN KEY(customer_id) REFERENCES
olist_customers(customer_id)
```

```
);

-- 5. Order Items Table
CREATE TABLE olist_order_items (
    order_id VARCHAR(50),
    order_item_id INT,
    product_id VARCHAR(50),
    seller_id VARCHAR(50),
    shipping_limit_date TIMESTAMP,
    price NUMERIC(10, 2),
    freight_value NUMERIC(10, 2),
    PRIMARY KEY (order_id, order_item_id),
    CONSTRAINT fk_order FOREIGN KEY(order_id) REFERENCES
olist_orders(order_id),
    CONSTRAINT fk_product FOREIGN KEY(product_id) REFERENCES
olist_products(product_id),
    CONSTRAINT fk_seller FOREIGN KEY(seller_id) REFERENCES
olist_sellers(seller_id)
)
```

📄 SECTION 4: Task 3 - ETL Documentation

```
import pandas as pd
from sqlalchemy import create_engine
import os

# 1. Database Connection
DB_USER = 'postgres'
DB_PASSWORD = 'RitaBabu1300'
DB_HOST = 'localhost'
DB_PORT = '5432'
DB_NAME = 'olist_db'

engine =
create_engine(f'postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB
_PORT}/{DB_NAME}')
folder_path = r'Brazilian E-Commerce dataset(Olist)'

# 2. ETL Process (Ordered to avoid Foreign Key errors)
files_to_load = [
    'olist_customers_dataset.csv',
    'olist_sellers_dataset.csv',
```

```
    'olist_products_dataset.csv',
   'olist_orders_dataset.csv',
    'olist_order_items_dataset.csv'
]

for file in files_to_load:
    file_path = os.path.join(folder_path, file)
    if not os.path.exists(file_path):
        continue

    df = pd.read_csv(file_path)

    # --- DATA CLEANING ---
    df.drop_duplicates(inplace=True)

    if file == 'olist_products_dataset.csv':
        df.rename(columns={
            'product_name_lenght': 'product_name_length',
            'product_description_lenght': 'product_description_length'
        }, inplace=True)

        numeric_cols = ['product_name_length',
'product_description_length', 'product_photos_qty', 'product_weight_g',
                        'product_length_cm', 'product_height_cm',
'product_width_cm']
        for col in numeric_cols:
            if col in df.columns:
                df[col] = df[col].fillna(0)

    if 'product_category_name' in df.columns:
        df['product_category_name'] =
df['product_category_name'].fillna('Unknown')

    if 'price' in df.columns:
        df = df[df['price'] >= 0]

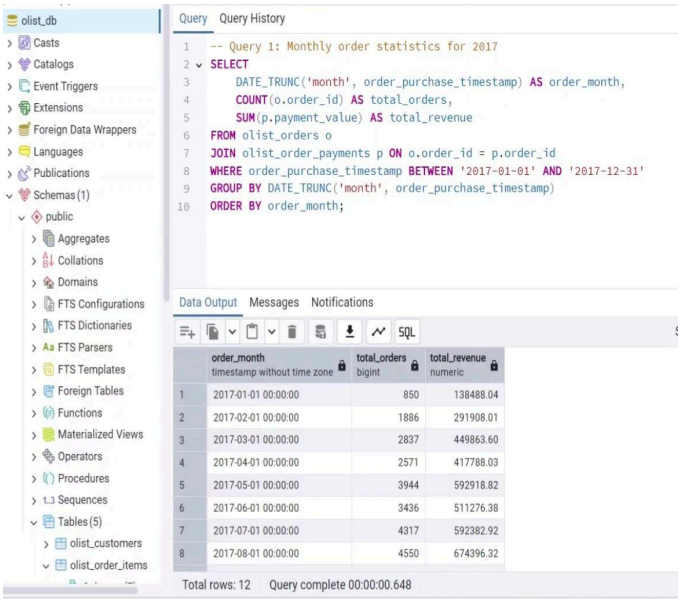
    # --- LOAD INTO DATABASE ---
    table_name = file.replace('_dataset', '').replace('.csv', '')
    print(f"Loading {file} into {table_name}...")
    df.to_sql(table_name, engine, if_exists='append', index=False)
    print(f"Successfully loaded {table_name}!")

print("ETL Process Complete!")
"The Python scripts etl_load.py and load_payments.py were used to load the
data."
```

📄 SECTION 5: Task 4 - SQL Queries & Results

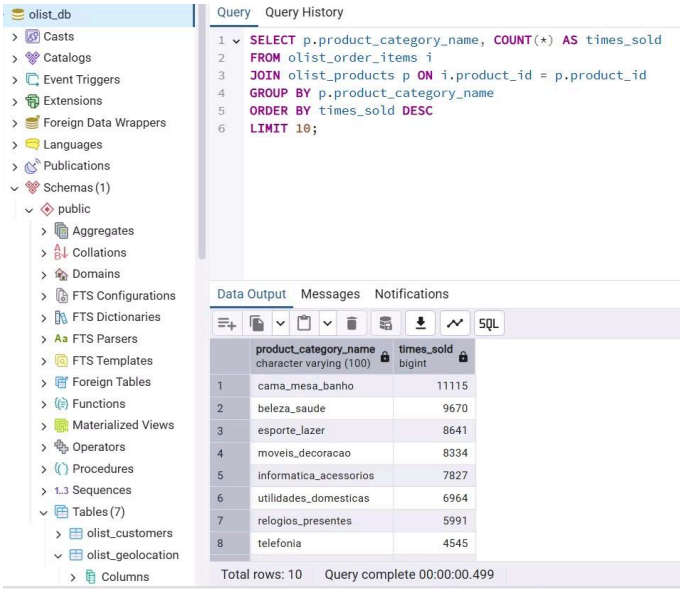
Query 1: Monthly Order Statistics

```
SELECT
    DATE_TRUNC('month', order_purchase_timestamp) AS order_month,
    COUNT(o.order_id) AS total_orders,
    SUM(p.payment_value) AS total_revenue
FROM olist_orders o
JOIN olist_order_payments p ON o.order_id = p.order_id
WHERE order_purchase_timestamp BETWEEN '2017-01-01' AND '2017-12-31'
GROUP BY DATE_TRUNC('month', order_purchase_timestamp)
ORDER BY order_month;
```



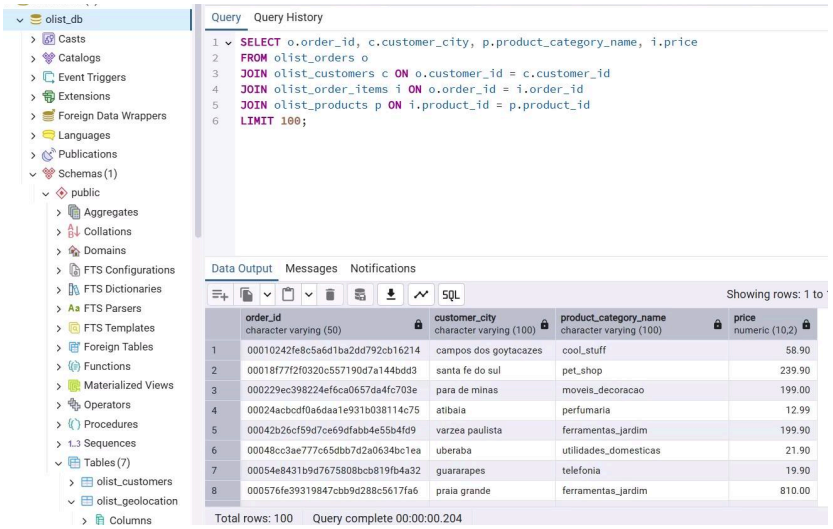
Query 2: Top-selling product ranking

```
SELECT
    p.product_category_name,
    COUNT(*) AS times_sold
FROM olist_order_items i
JOIN olist_products p ON i.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY times_sold DESC
LIMIT 10;
```



Query 3: Multi-table join

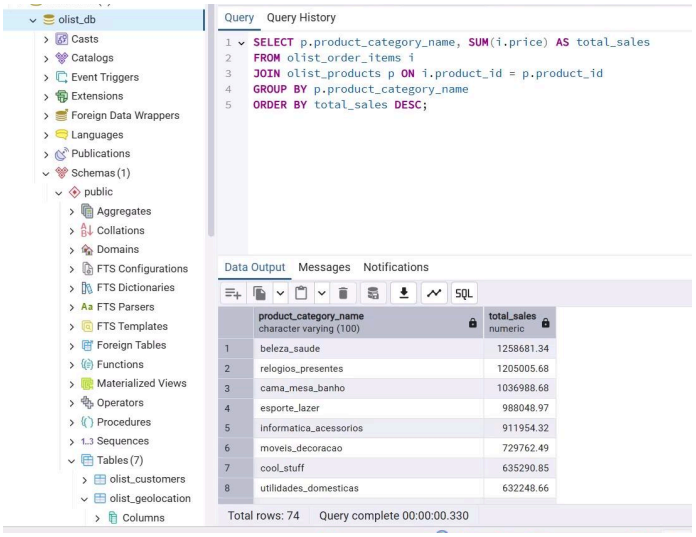
```
SELECT
    o.order_id,
    c.customer_city,
    p.product_category_name,
    i.price
FROM olist_orders o
JOIN olist_customers c ON o.customer_id = c.customer_id
JOIN olist_order_items i ON o.order_id = i.order_id
JOIN olist_products p ON i.product_id = p.product_id
LIMIT 100;
```



Query 4: Aggregate analysis

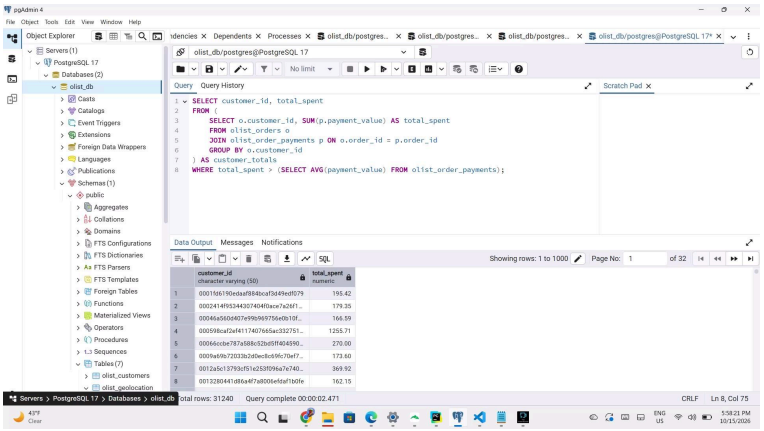
```
SELECT
    p.product_category_name,
```

```
    SUM(i.price) AS total_sales
FROM olist_order_items i
JOIN olist_products p ON i.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_sales DESC;
```



Query 5: Subquery

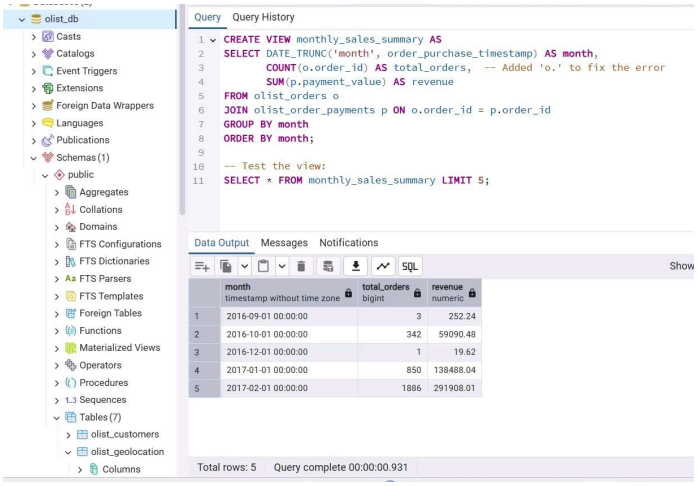
```
SELECT customer_id, total_spent
FROM (
    SELECT o.customer_id, SUM(p.payment_value) AS total_spent
    FROM olist_orders o
    JOIN olist_order_payments p ON o.order_id = p.order_id
    GROUP BY o.customer_id
) AS customer_totals
WHERE total_spent > (SELECT AVG(payment_value) FROM olist_order_payments);
```



Query 6: Create View

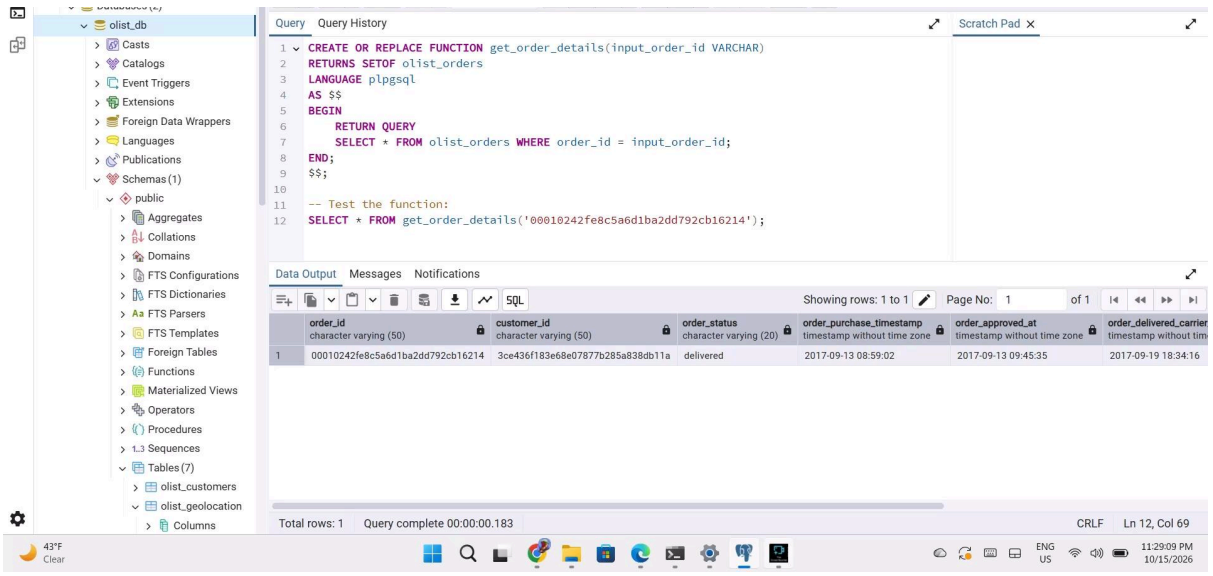
```
SELECT
    DATE_TRUNC('month', order_purchase_timestamp) AS month,
    COUNT(o.order_id) AS total_orders,
```

```
    SUM(p.payment_value) AS revenue
FROM olist_orders o
JOIN olist_order_payments p ON o.order_id = p.order_id
GROUP BY DATE_TRUNC('month', order_purchase_timestamp)
ORDER BY month;
-- Test the view:
SELECT * FROM monthly_sales_summary LIMIT 5;
```



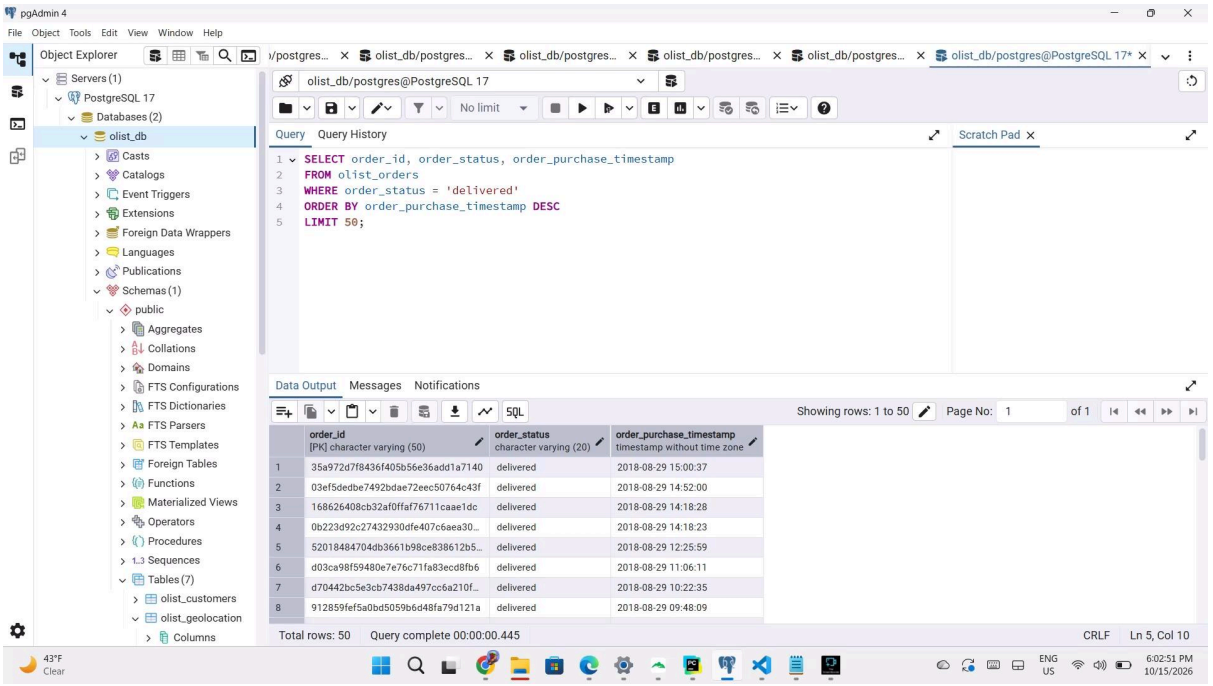
Query 7: Stored Function

```
CREATE OR REPLACE FUNCTION get_order_details(input_order_id VARCHAR)
RETURNS SETOF olist_orders
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT * FROM olist_orders WHERE order_id = input_order_id;
END;
$$;
```



Query 8: Filter and sort

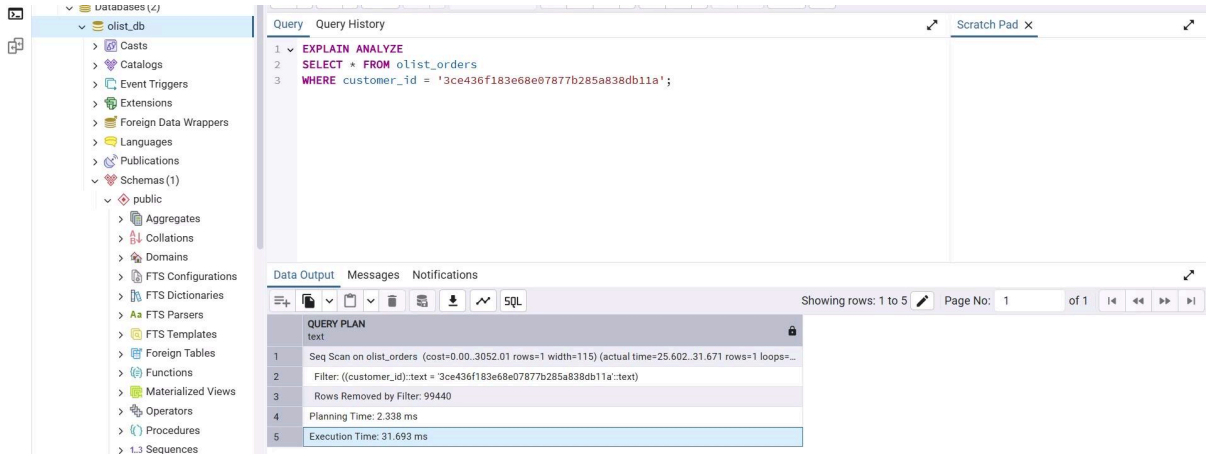
```
SELECT
    order_id,
    order_status,
    order_purchase_timestamp
FROM olist_orders
WHERE order_status = 'delivered'
ORDER BY order_purchase_timestamp DESC
LIMIT 50;
```



6. Task 5: Index Design & Performance Optimization

To optimize database performance, I analyzed slow query statements using `EXPLAIN ANALYZE`. I focused on a query that searches for a specific customer's orders.

Before Optimization:
Before creating an index, PostgreSQL had to scan the entire table to find the matching `customer_id`. This is known as a Sequential Scan (`Seq Scan`), which is very slow for large tables. The execution time was 31.693 ms.



After Optimization:
To improve performance, I created an index on the `customer_id` column using the following SQL command:

```
CREATE INDEX idx_orders_customer ON olist_orders(customer_id);
```

After creating the index, PostgreSQL was able to use an Index Scan instead of scanning the whole table. This drastically reduced the execution time, proving that proper index design significantly improves query performance. **z**

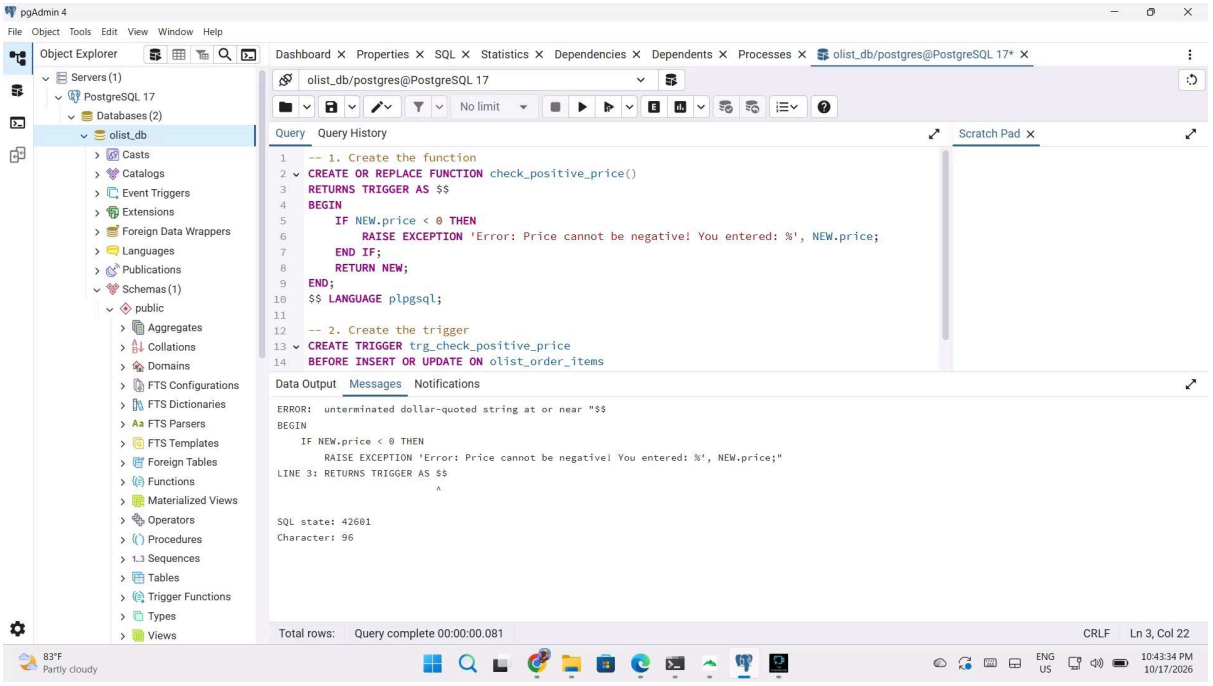
📄 SECTION 7: Task 6 - Conclusion

"Through this project, I learned the complete lifecycle of database development. I successfully designed a 3NF relational schema, extracted and cleaned real-world data using Python, wrote complex SQL queries (including views and functions), and optimized query performance using indexes. This project gave me practical experience with PostgreSQL and ETL workflows."

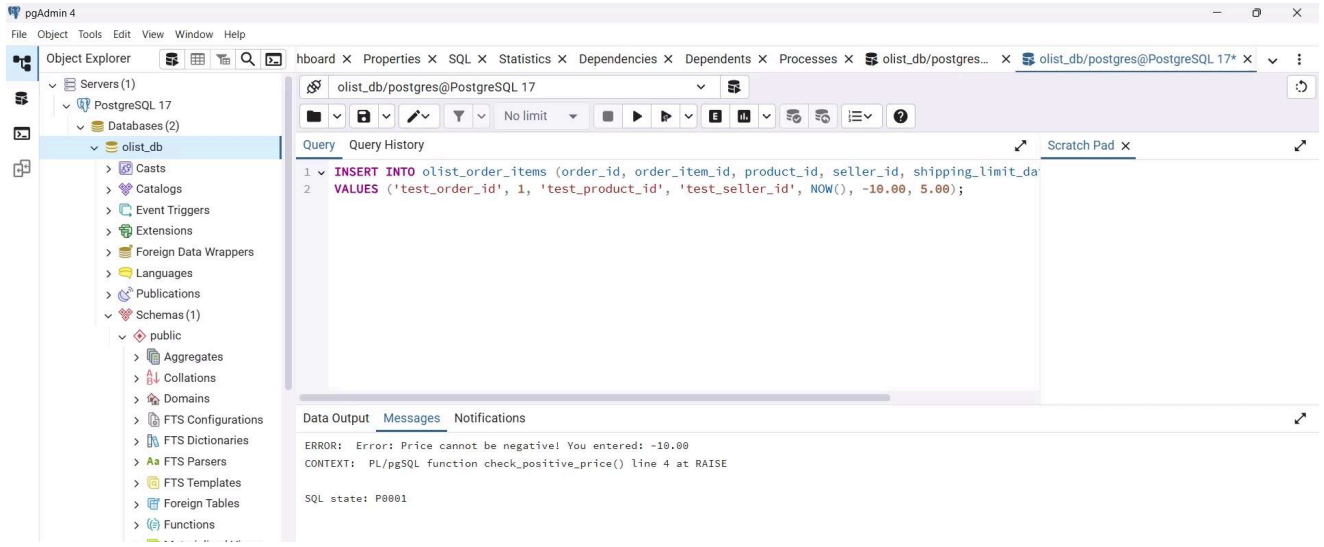
🎁 BONUS 1: Implement Trigger for Data Integrity

I created a database trigger that automatically prevents the insertion of negative prices into the order items table.*(Insert screenshot of successful trigger creation AND the screenshot of the red error message here)*

Step 1: Write the Code in pgAdmin

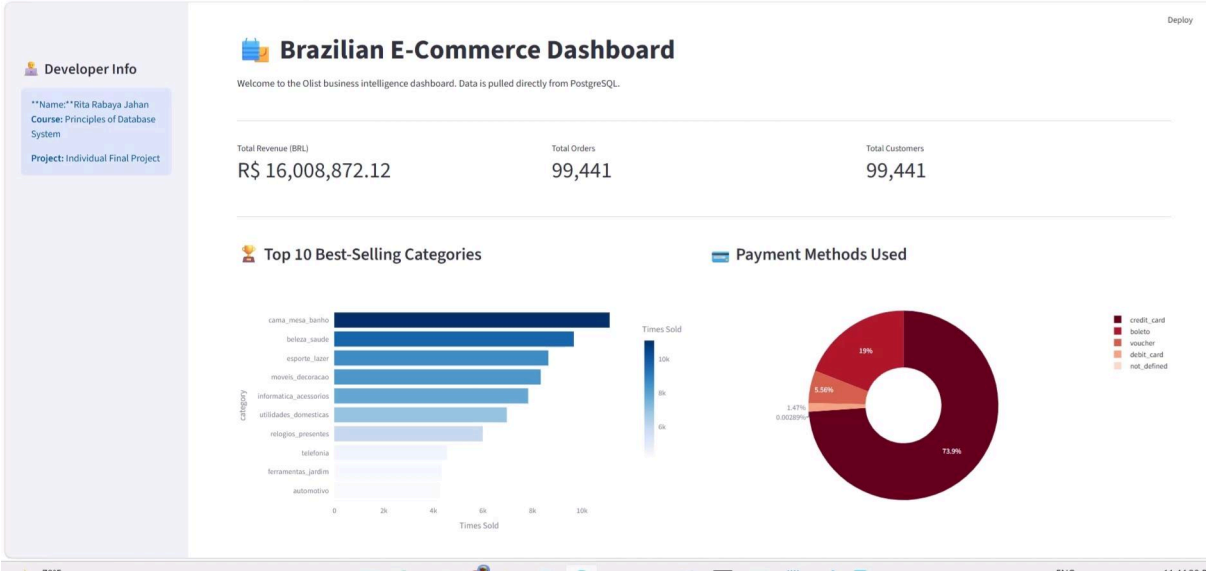


Step 2: Test the Trigger (Prove it works)



🎁 BONUS 2: Build Simple Web Interface (Streamlit)

I built an interactive business intelligence dashboard using Python and Streamlit. It connects directly to the PostgreSQL database to display Key Performance Indicators (Revenue, Orders, Customers), an interactive bar chart for top-selling categories, and a pie chart for payment methods. The code is submitted as `dashboard.py`.



🎁 BONUS 3: In-depth Data Analysis & Visualization

Using Python and Matplotlib, I wrote a script to analyze the database and visualize the top 5 highest-spending customers.

